JACARANDA LABS PRESS

HARARE · 2026

The Ledger and the Shopkeeper

Building commerce for the markets the platforms forgot

COURAGE GARIKAI

JACARANDA LABS · FIRST EDITION



CONTENTS

About this book 4

Who it is for 6

How to read it 7

A note on evidence 8

Part One — The Gap 9

1 The forty-tab panic 10

2 Why the obvious answer is wrong 14

3 The record is the product 18

Part Two — The Machine 22

4 A shop in one link 23

5 The statement that can be recomputed 28

6 Quotas are a design material 33

7 Finding the first hundred 37

8 The number that could not be dialled 43

Part Three — The Rules 46

9 Never hold the money 47

10 Never lie in the copy 52

11 Never post to your customers 56

12 Two systems, one sheet 59

Part Four — The Craft 63

13 What broke, and what it taught 64

14 The uncomfortable arithmetic 68
Appendix A · The statement specification 71
Appendix B · The admission gate 72
Appendix C · The reserve ladder 73
Appendix D · The failure taxonomy 74
Appendix E · The three commitments 75
A short reading list 76
Colophon 77

About this book

This is a book about a piece of software and the idea underneath it. The software is called Tapstall. The idea is that the thing a small merchant in a hard economy needs from the internet is not a wallet. It is a record.

I built Tapstall in 2026, in Harare, with no capital, on free infrastructure, for merchants in Zimbabwe, South Africa, Kenya and Nigeria. It has never held a customer's money and it never will. What it produces instead is a statement: a page a shopkeeper can hand to her accountant that says what she sold, what she was paid, and what she is still owed. Everything in this book grows out of that one decision — refuse custody, deliver the record — and out of what it cost to hold that decision while the software was being built.

The book is in four parts. **Part One** describes the gap: why the obvious remedy for a merchant without records is the one remedy she cannot safely accept. **Part Two** is the machine: the storefront, the order, the statement, the budgets, the search for merchants. **Part Three** is the rules — the three commitments that constrain every line of code and every sentence of copy, and what happens when you take them seriously. **Part Four** is the craft: integration, failure, and the uncomfortable arithmetic of a first customer.

I have tried to write the book I wanted when I started: specific enough to be argued with, honest enough to be useful, and free of the two genres that surround this subject — the triumphant thread about a machine that built a busi-

ness overnight, and the warning that says nothing about how to build one that behaves.

Who it is for

For the person building commerce software for markets that are not Silicon Valley: the agency owner in Nairobi, the developer in Bulawayo, the accountant who keeps eleven sets of books, the operator who wants to resell a rail to her own clients without becoming a payments company. Also for the product engineer anywhere who has been asked to "just add payments" and suspects the request is the wrong one.

How to read it

Straight through, if you want the argument. If you want the machinery, start at Chapter Four. If you have been handed an integration and told to make it work with code you do not own, Chapter Eleven is the one to read first. Chapter Twelve is a post-mortem in five movements and can be read as a warning at any point.

A note on evidence

Every number in this book comes from the running system or from its public history: the release chain, the test suite, the production endpoints, the records in the database. Where I quote an error, I quote it as it was printed. Where a figure is a design ceiling rather than an observation, I say so. Nothing here is rounded up to make the software look better than it is.

PART ONE

The Gap

CHAPTER 1

The forty-tab panic

At seven in the morning a shopkeeper in Harare opens her laptop and finds forty tabs. There is the WhatsApp Web tab where the orders are. There is the EcoCash message thread where some of the payments are. There is a bank statement she downloaded on Tuesday and has not opened. There is a spreadsheet called `stock final FINAL v3` and another called `stock final FINAL v3 (2)`. There is a Facebook page with three unread enquiries, a phone with two missed calls from a number she does not recognise, and a delivery note photographed sideways by her brother.

She is not disorganised. She is a competent trader running a real business with real customers, and she is doing it with a payment rail that produces records for the mobile network operator and an order channel that produces records for nobody.

By half past seven she will make a decision about which of these tabs to close, and the decision will be made on the basis of what feels urgent rather than what is expensive. The expensive thing is invisible: nobody in that stack of tabs can tell her, quickly and reliably, **which of last week's orders were actually paid**. She has the information, in principle — it exists in the EcoCash thread, in the cash she took, in the cousin who delivered — but she cannot assemble it. The information is present and the knowledge is absent, and the distance between those two things is the subject of this book.

What she is actually buying

Ask a software company what this merchant needs and you will hear: a storefront, a payments integration, maybe an inventory feature, ideally an app. Ask the merchant, and the words will be different. She will say she wants to stop losing track. She will say she is not sure whether a customer owes her. She will say that at month-end she and her accountant spend two days reconstructing a month that already happened.

None of that is a request for a wallet. All of it is a request for **reconciliation** — the ability to compare what should have happened with what did, and to act on the difference. Reconciliation is not a payment feature. It is a bookkeeping property, and it is produced by records, not by funds.

This distinction has a commercial consequence that took me a long time to see clearly. If the merchant's problem is funds, then a platform must touch funds, and touching funds is expensive, regulated, and risky. If her problem is records, then a platform can touch records only — which is cheap, unregulated in the ways that matter, and — crucially — **safer for her**.

The three tabs that matter

Strip away the forty and there are three. There is the **order**, which is a promise: this customer wants this thing at this price. There is the **payment**, which is a fact that happens outside your software. And there is the **connection** between them, which is the thing nobody owns. Every commerce product is, at bottom, an opinion about that connection. Point-of-sale systems own it by owning the till. Marketplaces own it by owning the buyer. Payments com-

panies own it by owning the money. Each of those is a way of forcing the connection to be visible by taking custody of one of its ends.

There is a fourth way, and it is the one this book is about: **ask both parties to say what happened, record both, and compare them.** No custody. No till. No marketplace. Just a record that is two-sided, and an artefact that does the comparing.

That is a thinner business than owning payments. It is also a business you can start this month, in a market the platforms have decided is not worth the compliance cost, with a merchant who will pay a flat fee because you have removed two days of her month.

The panic is the finding

I have described the morning because it is the empirical starting point of everything that follows, and because it is routinely misread. A developer watching that shopkeeper will conclude she needs organisation. An investor will conclude she needs software. A payments company will conclude she needs access. All three are wrong in the same direction: they treat the absence of records as a defect in the merchant rather than as a missing product.

She is not failing to keep records. The record-keeping layer she has been given — a chat application, a wallet with a statement she cannot query, and a bank — was not designed to produce the one document her business runs on. Nobody built it. That is a product gap, and product gaps in unglamorous places are where a small team can still do something that matters.

> **The first rule.** A merchant's problem is not that money moves badly. It is that nothing survives the movement.

CHAPTER 2

Why the obvious answer is wrong

The obvious answer is a payments platform. It is obvious because it is what every commerce company in the wealthy world did, and because it appears to solve the problem at its root: if you own the money, you own the record of the money, and the record is free.

It is wrong here, for three reasons that compound.

Custody is a promise you cannot keep

The moment a platform accepts money on a merchant's behalf, it is holding funds it must later hand over. That is custody, and custody is a regulated activity everywhere for a reason: the holder can fail. In economies with fast, cheap and reliable commercial recourse, a failed intermediary is an inconvenience with a queue of creditors. In economies where recourse is slow, expensive, or contested — and Zimbabwe's cash-scarce, multi-currency, high-inflation-memory economy is the paradigm — a failed intermediary is the end of the merchant's month.

Consider what the merchant is being asked to accept. She is a trader with a few hundred dollars of daily turnover, thin margins and no legal department. She is asked to route that turnover through a company she met last month, which holds it for a settlement period, on the strength of a dashboard and a promise. Every additional day between her customer's payment and her own cash is a day she is an unsecured creditor of a startup.

She will often accept this, because she has no alternative, and that is precisely what makes it an ethical problem rather than a market outcome. The alternative is to make the platform's solvency irrelevant to her, and the only way to do that is to never hold her money.

The percentage is a tax on competence

The second reason is the pricing model that custody makes natural. If you hold the money, you can see the volume, and if you can see the volume you will charge a share of it. One and a half percent sounds modest until you compute it against a margin that is often under ten percent. For a merchant selling tills and scales in Harare, a percentage fee is not a service charge; it is a partner who takes a cut of every good month and nothing in a bad one, which is exactly backwards from the risk she is carrying.

There is a second-order effect that matters more. Percentage pricing makes a platform's revenue a function of **volume**, which makes the platform's interests diverge from the merchant's in two specific ways. It rewards pushing orders the merchant cannot service, and it rewards a checkout that is frictionless in the direction of the payment rather than cautious in the direction of the record. Neither of those is a conspiracy; both are gravity.

A flat fee inverts this. The platform's revenue is a function of **retention**, and retention in this market is a function of whether the merchant's month-end got easier. That is a healthier alignment, and it is only available to a platform that is not pricing the transaction.

Capability without custody

The third reason is the simplest and the least believed: nothing about the merchant's actual problem requires the platform to touch money. She is not short of a recipient. She already has a wallet, a bank account, a cousin with a phone, and a drawer of cash. She is short of a **record that both she and her customer will accept**, and a record is a document.

Once you see this, the architecture falls out almost automatically. Let the buyer order through a link. Let the merchant confirm the order. Let the buyer say what they paid and how. Let the merchant mark what actually arrived — on their own rail, in their own account — and let the platform compare the two claims and print the difference. At no point does the platform need to be a party to the money. It needs to be a party to the sentence about the money, and those are different jobs with different costs.

What this costs you

It would be dishonest to present this as free of trade-offs. Refusing custody costs three things.

You cannot prove payment. If money never passes through you, you cannot assert that it moved. You can only compare two statements made by two parties. That is a genuine loss of certainty, and the correct response is not to hide it but to make the ambiguity small, explicit, and visible — which is what Chapter Five is about.

You cannot earn float or spread. That is a real revenue stream, and refusing it means your business must survive on fees alone. For a small platform that is fine; for anyone

who has raised money on the expectation of payments economics it is not.

You cannot be the checkout. Without custody you are not the thing a buyer trusts with a card; you are the thing that points at the thing the buyer already trusts. That makes you less exciting to investors and less sticky in the way payments companies are sticky. It also makes you easier to adopt, because a merchant does not have to move her money to use you.

Three costs, accepted deliberately. What is bought with them is described in the next chapter, and it is the reason this book exists.

> **The second rule.** If your business model needs the merchant's money to be visible to you, you have chosen a business that is priced against her success and exposed to your own failure.

CHAPTER 3

The record is the product

Here is the sentence this book argues for, in full:

> A merchant in a low-infrastructure economy will pay a flat fee for a document that tells her, every evening, exactly what she sold, what she was paid, and what she must chase — and she will pay it because the company that produced it never touched her money.

There are three claims inside that sentence and each can be tested separately. She will pay a flat fee: that is a pricing claim, and flat pricing is what makes the product affordable and the incentives straight. She wants the document: that is a product claim, and it is falsifiable — if merchants ignore the statement, the thesis is wrong. And she values the fact that the platform does not touch her money: that is a trust claim, the hardest to measure and the one that matters most in a market where institutions fail visibly.

Why a document

Software people undervalue documents because they are static. A document cannot be queried, integrated, or built on; it is a dead artefact that someone has to read. This is precisely why it works.

The shopkeeper's month-end problem is not a data problem. She has the data. It is a judgement problem: out of a month of noise, which four facts matter? A dashboard answers that by asking her to construct the answer from widgets. A document answers it by saying it: **eleven orders,**

nine matched, two to chase, forty dollars outstanding, fees seven, net two hundred and sixteen.

The document is also a social object. In Star and Griesemer's phrase, a boundary object: solid enough for the shopkeeper ("chase these two"), structured enough for the accountant ("these nine reconcile, these two do not"), and stable enough to pass between them without an argument about what it means. Most commerce software fails at exactly this seam. It produces dashboards for the merchant and exports for the accountant, and the two never agree, and the disagreement is discovered in the worst possible conversation — the one about money.

A single document, generated from a single computation, dissolves that. It is a humble thing to build a company on. It is also the only thing in this space that both parties will actually look at.

The three sentences you may never blur

Any document about money is a promise about language. Three distinctions do the work, and every failure I have seen in this domain is a failure to hold one of them.

Matched is not paid. "Matched" means the merchant's record of receipt agrees with the order's record of amount. It does not mean the platform verified a payment, because the platform was not there. A statement that says "paid" when it means "matched" is a lie with a spreadsheet behind it, and it will be discovered at the worst moment.

Expected is not owed. An expected order is a promise the merchant is holding open. Calling it a debt changes the emotional register of the document and, in some jurisdictions, its legal character. The merchant knows the differ-

ence between a customer who will pay on Friday and a customer who is avoiding her. The software does not, and must not pretend to.

To chase is the actionable number. Of the six figures on the statement, five describe the past. One describes the work: the sum that is claimed but not confirmed. If the document makes only one number large, that is the one, because it is the only one a person can act on before lunch.

I have this rule written on a page in the repository, and a test asserts that the statement page and the statement API produce identical figures for the same day. The test is not about arithmetic. It is about the promise: one computation, so the page can never say something the file does not.

What the product is not

It is not a wallet, a bank, an escrow, or a marketplace. It is not a till, so it will not count your cash drawer; it is not an ERP, so it will not do your VAT. It is a thin layer of order and evidence that sits between the channels a merchant already sells on and the bookkeeping she already pays for.

Thinness is a strategy, not an apology. Thin products can be adopted in an afternoon, priced in a sentence, and run on free infrastructure by one person. That last property is not incidental: it is what allows the product to exist in markets where the customer's willingness to pay is seven dollars a month. A platform with a sales team and a compliance department cannot serve a seven-dollar customer. A thin platform can, and thinness is the only reason the economics work at all.

The test of the thesis

If the record is the product, then three things should be true, and they are the things I would ask any reader to check against their own market.

First, merchants should ask for the statement before they ask for the storefront. In practice they ask for the link first and the statement second, and the second request is the one that tells you the product has landed, because it is the request made by someone who has started to rely on you.

Second, the statement should be shown to a third party. A document nobody forwards is a dashboard with better typography. The measure of success is the accountant's email asking for last month's statement in the same format.

Third, and least comfortably, the merchant should tell you that she likes that you do not hold her money. That sentence is the whole thesis in the customer's own words, and if you never hear it, you have built a generic storefront with an unusual pricing page.

> **The third rule.** Build the document first. Everything else in commerce software is a delivery mechanism for it.

PART TWO

The Machine

CHAPTER 4

A shop in one link

The first thing Tapstall does is give a merchant a shop that costs nothing to open and nothing to explain.

A merchant registers with a shop handle, an owner name and a password. She publishes her catalogue as a small table — nine columns, semicolons optional, the kind of thing she can paste from the spreadsheet she already keeps — and the platform renders a storefront at `tapstall.zlnew.workers.dev/s/<handle>`. The storefront is server-rendered HTML with no script. It loads on a two-year-old Android phone on a metered connection, because that is what the buyer has.

Then she shares one link. In a WhatsApp thread, on a Facebook page, in a bio, on a receipt, in a broadcast list. The link is the product. It replaces the fifteen-message conversation — what do you have, how much is it, do you deliver, do you have a photo, can you send your number — with a page that answers the first four questions and takes the order.

The order is a sentence, not a session

A buyer opens the storefront, picks items, and places an order. There is no account, no password, no email verification. The buyer gives a name and a number and, if they have one, an address. That is the entire form, and it is short because every additional field is a reason a buyer on a metered phone will close the tab.

The buyer then sees one page: the order, the amount, and a single button. Pressing it does not take money. It posts the order to the merchant and produces a reference the buyer can keep. What happens next is a conversation on the channel the buyer already trusts — a knock on the door with cash, a mobile-money transfer, a card on a link the merchant chooses. Tapstall's job in that moment is to have made the order durable and legible, not to have inserted itself between two people who are about to exchange money.

This design decision — the buyer's confirmation is a record, not a transaction — is the load-bearing wall of the entire architecture. Everything downstream follows from it. It means the system can run on free infrastructure, because it is doing arithmetic on small records rather than processing payments. It means it can be trusted by a merchant who has never met me, because the merchant can see that the software has no way to move her money. And it means the platform's obligations are bookstore obligations: keep the record, show it to the right person, and never lose it.

Two-sided from the first minute

The most important property of the one-tap confirmation is not convenience. It is that it makes the record **two-sided**.

An order that exists only in the merchant's dashboard is a private claim. An order that exists in the buyer's browser and in the merchant's dashboard is an agreement, with the same amount and the same items written on both copies. When those two copies are later compared with the merchant's own record of receipt, you get the three-way com-

parison that makes reconciliation possible without any party holding anyone's money.

This is why Tapstall's confirmation page carries a capability token in the URL rather than a session cookie. The buyer has no account, so the link is the credential, and it is scoped to exactly one act: confirm this order. It cannot read other orders, cannot change an amount, cannot reach another shop. The same pattern applies to the proof-of-payment path — a separate token, a separate page, a separate capability — so that a link sent to a buyer can never do an owner's job.

The engineering here is unremarkable, but the consequence is not. A merchant can forward her own order link to a customer who will press it on a phone with no account, and the resulting record is as good as one created by a logged-in user. That is what makes adoption possible in a market where account creation is a serious barrier and passwords are a serious hazard.

The merchant's morning

The merchant's view is a dashboard with one screen that matters. At the top, today's orders with their state. Beside that, the day's settlement statement. Below, the catalogue, the storefront link with a copy button, and an export.

The dashboard is deliberately not a control panel for the platform. It has no analytics about the platform's audience, because there is no audience — the merchant's customers are the merchant's. It has no upsell banner for a higher tier, because the merchant came for a record and a banner about pricing in the middle of her morning work is a small

act of disrespect. It shows what she needs to do the day's work and then gets out of the way.

Two links sit at the bottom of the page, and both exist for the same reason: the record must be shareable. One is the books export — a CSV of the period, for a spreadsheet. The other is the settlement statement — a page, for a human. They are different formats of one computation, and if they ever disagree, the build fails.

Why a link and not an app

Every merchant I have spoken to asks, eventually, for an app. The answer is that a link is better, for now, and the reasons are all about the buyer.

An app requires an install, which is a decision, which is a drop-off. A link requires a tap, which is not. An app requires a store listing, a review cycle, and a version of Android you may not control. A link requires a browser, which every phone already has. An app can send push notifications and hold local state; a link cannot, and for this product neither is needed, because the notification the merchant cares about travels over WhatsApp anyway.

There is a deeper reason. An app would let the platform hold state on the buyer's device, and holding state on a buyer's device is the first step toward holding something else. A link that produces a record and then forgets is a smaller surface, and a smaller surface is easier to audit. When the whole promise of the company is we hold nothing of yours, the product should look like a company that holds nothing, even in the small things.

> **Rule four.** Make the buyer's confirmation a record, not a transaction, and the rest of the architecture will follow.

CHAPTER 5

The statement that can be recomputed

This is the chapter the whole book points at. If you build nothing else from it, build this.

Six numbers

The settlement statement reports six figures for a single day:

Figure	Meaning
Expected	the sum of confirmed orders for which no merchant-side receipt exists
Matched	the sum of orders where the merchant's record of receipt agrees with the order
To chase	the actionable number: the outstanding part of expected
Refunded	orders reversed before settlement
Fees	the platform's flat charge, never a share of volume
Net	matched minus fees

Underneath, a table by rail: cash, mobile money, bank transfer, card, each with the count of orders and the amount outstanding. Above, one sentence of instruction, generated from the numbers, naming the single thing to do next. If the day is empty, the page says so in words — nothing on this day — because a document full of zeros reads as a failure to a merchant, and a quiet Sunday is not a failure.

Why it can be trusted by someone who does not trust you

The statement's authority comes from one property: **it is a function of the event log and nothing else.** Give a sceptical accountant a copy of the platform's records for a day and a description of the rules, and she can recompute every number on the page. She does not have to believe the platform. She has to believe arithmetic.

That property sounds obvious and is frequently violated. Most commerce dashboards are functions of a mutable state machine: an order's status field is edited in place, history is lost, and the number on the screen depends on which version of the row you are looking at. A statement generated from such a system cannot be recomputed by anyone outside it, because the inputs no longer exist. The document then functions as an assertion rather than as evidence, and assertions about money do not survive contact with an accountant.

The discipline that fixes this is unglamorous: **every change is an event, and every number is derived.** An order is created, confirmed, marked received, refunded — four events with timestamps and actors. The statement is a fold over the day's events. Nothing is updated in place; nothing is deleted. Storage is append-mostly, and where a row must change for performance, the derivation still runs over the event stream so that the two can be compared.

The two doors, one arithmetic

Tapstall exposes the statement twice. As a **page** a merchant can read, print, or hand over — server-rendered, no script, a print stylesheet, `no-store`, reachable only by a live session. And as **data** — an API returning JSON for a ma-

chine, CSV for a spreadsheet. Both call one implementation. A test asserts they agree for the same day.

Two doors, one arithmetic, and a test that fails the build when they diverge. This is the cheapest possible defence against the most expensive class of bug in a reconciliation product: the page that says one thing while the file says another, discovered by the accountant, in front of the merchant, at month-end.

Reachable, but not public

A statement about a merchant's takings is the most sensitive document the platform produces, and it must be reachable exactly once: by the merchant, in a live session. The access rules are boring and absolute.

No session: the request is redirected to the door with the shop named, so a merchant who has been logged out knows what happened without being shown a figure. A dead session: a page that says the session no longer works and that **nothing about the money changed** — because the fear a merchant has when a money page fails is that the money failed with it. No such shop: a page that says so, and nothing else. Every one of those responses carries `no-store`, and none of them contains a script tag.

That last detail is not purism. A statement page with a script is a page whose contents can be altered by whatever the script loads, which means a merchant's books can be edited by a third party without the platform's knowledge. The page that reports money should be the least dynamic page in the system: static markup, server-rendered numbers, no execution.

Day navigation, and the quiet-day rule

The statement takes a date and lets the merchant move backward and forward. The morning routine this creates is small and, in the merchants who have used it, the whole value of the product: open today, look at to chase, hand the phone to whoever is counting the cash.

The quiet-day rule is the least technical design decision in the book and possibly the most important. On a day with no orders a naive implementation renders a table of zeros. A merchant reads a table of zeros as something is wrong. So the page says, in words: nothing on this day; a statement with zeros in it is not a statement that something went wrong.

Small mercies compound. A product that reassures correctly on the empty case is a product a merchant will open on a Sunday.

The specification, as a promise

Four clauses, which together define a conforming statement:

Any statement must report the six figures, the per-rail breakdown, and one sentence of instruction. It must be computed from the day's events alone, so a third party can recompute it. It must contain no script, carry `no-store`, and be reachable only by a live session of the merchant it describes. And it must never describe a matched order as paid, an expected order as owed, or the fee as a share of volume.

That last clause is not decoration. An implementation that meets the first three and violates the fourth is a competent piece of software doing a dishonest job, and the clause ex-

ists so that such an implementation is non-conforming rather than merely disappointing.

> **Rule five.** If a number cannot be recomputed from the record by someone who does not trust you, it is not a fact. It is a claim.

CHAPTER 6

Quotas are a design material

Tapstall runs on free infrastructure: a Cloudflare Worker, two key-value namespaces, and a build that ships the same bytes the tests ran against. The marginal cost of the platform is zero. The price of zero is quota, and quota is the material this chapter is about.

Five budgets that do not bind equally

The free tier is not one limit. It is several: a hundred thousand worker requests, a hundred thousand key-value reads, a thousand writes, a thousand deletes, a thousand lists, a gigabyte of storage, per day. The first two are roomy — a small platform would have to be very popular to exhaust them. The last four are not. A thousand writes a day is a real constraint on a system that records orders, security state, operator saves, scheduled automation and telemetry.

The interesting engineering question is not how to stay under a thousand writes. It is **what to do when the thousandth write of the day is the wrong one**. A naive system spends writes in arrival order, which means the thing that happens to run at 23:50 — a marketing digest, a telemetry roll-up — can be the thing that prevents an order from being recorded. That is not a performance bug. It is a design failure with a moral shape: the least important part of the system starved the most important part.

The reserve ladder

Tapstall's answer is a priority ladder with reserves. Every write is classified — money, security, operator, automation, telemetry — and each class must leave a reserve behind it for the classes above:

Class	Keys	May spend up to
Money	`order:`, `store:`, `sale:`, `paid:`, `tx:`, `renew:`	1000
Security	`lock:`, `pin:`, `rl:`	975
Operator	`audit:`, `img:`, `tpl-ovr:`, `digest:`	900
Automation	`press:`, `mesh:`, `lead:`, `seller:`, `cron:`	800
Telemetry	everything else	700

The arithmetic is one line: a write from class p is admitted if the writes already spent, plus one, is at most the ceiling minus the reserves of every class above p. Telemetry is additionally paced through the day, so that an isolate waking at one in the morning cannot burn the soft ceiling just because it woke up first and felt industrious.

The theorem worth having

From that one line follows a guarantee worth stating precisely, because it is the sort of guarantee a merchant can feel even if she never reads it.

An order write can fail for lack of budget only if all thousand writes of the day are spent.

The proof takes four lines. The money class has a reserve of zero above it, so its admission condition reduces to

spent plus one, at most a thousand. Every lower class must stop at least twenty-five short of the ceiling. Therefore no amount of automation, operator work or telemetry can bring the total to a thousand; at least twenty-five writes always remain for money. A marketing run cannot be the reason an order fails to land.

This is worth building because it converts a value into arithmetic. "Orders are more important than marketing" is a sentiment that survives exactly as long as the person holding it is awake. "Automation may not exceed eight hundred writes" is a constraint that survives the person.

Refusals should be legible

The second half of the discipline is that a refusal must explain itself. Tapstall's write accounting returns one of three reasons: within budget, reserved for higher priority, or paced for later in the day. A developer reading that in a log knows immediately whether they are looking at a capacity problem, a priority decision, or the clock. A generic quota exceeded tells them nothing and costs them an afternoon.

The same principle applies to the discovery system in Chapter Seven. A pass that files nothing must say whether it found nothing or was not allowed to look. A day with no candidates and a day with no permission are the same number and completely different facts.

What quota shapes

Designing under quota changes the product, and mostly for the better. Memory first, reads second, indexed lists third, and a write only for a fact that must survive the isolate — that ordering produces a system with fewer writes, simpler failure modes, and better behaviour under load

than one designed against a generous budget. It also produces a system whose costs are legible: when something gets expensive, you know which class it belongs to and therefore whose work it is displacing.

There is a limit to this virtue. Zero marginal cost buys you the ability to exist in a seven-dollar market; it does not buy you scale, and a platform that succeeds will need to spend money on infrastructure sooner than its architecture suggests. The honest formulation is that free infrastructure is a starting position, not a strategy, and the reserve ladder is worth keeping even after you are paying the bill — because it encodes an ordering of importance that has nothing to do with money.

> **Rule six.** Reserve capacity for the highest-consequence write, and make the arithmetic of the reservation explicit enough to test.

CHAPTER 7

Finding the first hundred

Every platform has a cold-start problem, and the standard solutions — ads, partnerships, a sales team — are all priced for markets where a customer is worth hundreds. In a market where a merchant pays seven dollars a month, acquisition has to be nearly free, which means it has to be found rather than bought.

This chapter is about that search, and about the specific failure it produced, because the failure is more instructive than the success.

The first attempt had no gate

The first version of the discovery system was, in essence, a search query and a parser. Ask a search engine for "web design agency Harare", read the results, file the titles, and let a salesperson sort it out. The reasoning was that volume was the constraint: get enough names in the queue and human judgement will separate them.

Three weeks of that produced **forty-seven records and zero prospects**. The queue contained, verbatim:

WhatsApp Web. Download WhatsApp. WhatsApp Help Center. Order — definition of order by The Free Dictionary. ORDER | English meaning — Cambridge Dictionary. Ecocash Charges Calculator 2026 — USD | Econet. Chewy Partner Hub | Start Selling On Chewy.com. Uber Eats. Zimbabwe Tourism. Zimbabwe — United States Department of State.

These are not errors in the sense of a broken parser. Every one of those pages was correctly found for the query. They are a **category error**: the system was asked for businesses and answered with pages that mention the words. And because it filed them into a queue with a score beside each one, it produced the appearance of work — a list of names, ranked — which is more dangerous than an empty queue, because it consumes the scarcest thing a small company has, which is the attention of the person who is supposed to be selling.

The gate

The fix is an admission gate, and it is worth spelling out because the predicates are the design. A candidate is filed only if all seven hold.

Not a platform. The page is not a social network, marketplace, dictionary, news site, bank portal, or asset CDN. The list of rejected hosts is long and grows every time a new impostor appears.

Verified. The candidate's page was actually fetched and read. A result from a search engine is a claim about a page, not evidence of a business.

Local. The page names the market — a city, a country domain, or membership in a bounded map query. A directory listing is local by construction because the directory is itself local; a map record's locality is its query, not a sentence.

Named. The business has a name of plausible length. Not a sentence, not a URL, not "Home".

Not generic. The name is not a category noun. "Accountants" is not a business. "Moyo & Associates" is.

Not an institution. The name is not a council, ministry, bank, university, hospital — and, after a lesson I will describe in a moment, not a charity, orphanage, foundation, church or trust.

Contactable. There is a published number whose digit count is valid for that market, or an address, or a checkable URL.

Seven predicates, each one a way of asking the same question: could a person, reading this record, open the page and start a conversation? If the answer is no, the record is not a lead. It is noise with a score.

The lesson in the gate's own gap

One of the first live passes after the gate went in filed a record that read: House of Mercy Children's Lagos, Nigeria. It had passed every predicate — its page was read, it named the city, it was named, it was not generic, and it published a number. The institution list contained orphanage in one spelling; the page used another.

Two things follow, and both generalise. First, a gate is a list, and lists leak; the discipline is to extend the list from the miss, immediately, with the exact string that got through, so the same page can never be admitted twice. Second, and more useful: **the gate is where your company's ethics become mechanical.** Nobody wants to cold-call an orphanage about a commerce subscription. The rule that says so has to live somewhere a tired person cannot forget it, and the only such place is code.

Volume is not the gate's job

When the merchants were not coming and the queue was thin, the instinct was to loosen the gate. That instinct is wrong, and seeing why is the single most useful piece of product thinking in this book.

A conjunction of rejections raises precision and lowers recall. Loosening it does not give you more prospects; it gives you more rows. More prospects come from a different axis entirely: **more sources, and better questions asked of them.** The gate decides which of the things you found are real; the source list decides how many things you find at all.

When we finally understood that, the numbers changed by an order of magnitude without touching a single predicate. One directory in one country became four source classes across four markets: a Zimbabwean classifieds index with forty categories; Kenyan and Nigerian business directories that answer a phrase and then hand you a company page with a published number; and, separately, the open map, which knows a shop the directories do not.

A live pass over Kenya and Nigeria, using real services rather than fixtures, returned six Kenyan company pages and six Nigerian ones for a single phrase — web design — and eight candidates with dialable numbers in the queue by the end of it. All seven predicates held. Nothing was loosened.

Interleave, and refund failures

Two scheduling lessons came out of the same period, and both are general.

Interleave sources; do not stack them. The first planner ordered sources by priority and took the day's allowance from the top. When the top source was a directory that had moved and returned a 404, the entire day's budget went into one failing source and the map and the search leg were never asked. A pass that produced zero candidates while three working sources sat idle. The fix is round-robin over source and market, so every working source is asked in every pass.

Refund a failure, cap the reserve. A job that fails hands its attempt back; a job that answers with nothing keeps it. Then a bad morning means a longer pass, not an empty one — bounded at three times the daily attempt budget, with a circuit breaker of three consecutive failures so an outage cannot become a crawl.

The distinction between a refusal and an empty answer turns out to matter everywhere. A directory with no matching category is not broken; it answered. An empty map result for the middle of Harare is a fact about Harare. A transport error is a fault. Conflating them produces a system that either panics on quiet days or ignores outages, and most systems manage to do both.

What you are actually looking for

The output of all this is not a list of names. It is a list of **sentences a salesperson can open with**, each of which is checkable on the page it came from. Talks about orders on its site but gives a buyer no way to order — a link would finish the sentence. Already sells through WhatsApp, with no page that shows an item and its price. Has no website of its own, so every order starts with a typed message.

A hook that cannot be checked is a rumour, and a rumour is what a salesperson discovers is wrong in front of a stranger. The measure that matters is not how many leads you have. It is what fraction of your leads can be verified at their source in one click, by someone who was not there when they were generated. That fraction is the honesty of your pipeline, and it is worth more than volume.

> **Rule seven.** Gate on verification and grow recall with sources. A queue that cannot be checked is worse than an empty one.

CHAPTER 8

The number that could not be dialled

The smallest incident in this book has the widest lesson.

A Nigerian company page published its telephone number like this:

> +234 (0)730 177 1710

The trunk zero in brackets is a convention, not a digit. The real number is 234 730 177 1710. The parser, written by someone who had seen `+254 (0)7xx` and `+263 77x` and thought about prefixes in general rather than about the bracket in particular, saw a string that began with the country code and left it alone. The result was:

> 23407301771710

Fourteen digits. Passes a plausibility check. Cannot be dialled by anyone.

Why this class of bug is dangerous

A crash is a bug that announces itself. This was a bug that produced a plausible object: a record that looked exactly like every good record around it, in a queue that a salesperson would work from. It would have been discovered the way such things are always discovered — by a stranger hearing a wrong number, or a dead line, or silence — in the middle of a call, which is to say at the moment of maximum cost to the company's credibility.

I call this **representation drift**: the record is internally consistent and externally wrong. It is the characteristic

failure of systems that validate shape rather than meaning. A digit-count check validates shape. A round-trip check validates meaning: re-parse the stored value and see whether it reproduces itself, then check the length against the market's real numbering plan.

The three fixes

Normalise to one canonical form and refuse the rest. Every published number becomes a single dialable string with the country code and no trunk zero. Anything that does not survive normalisation is not stored as a phone number; it is dropped and the record is not filed, because a lead with a wrong number is worse than a lead with no number.

Test with the strings you actually saw. The regression test for this fix contains the exact production string, bracket and all, plus the Kenyan local form (`0726239033` → `254726239033`) and the Nigerian `0708` form. The test is not a specification of the numbering plan; it is a fossil of a real failure, and fossils are more durable than specifications.

Treat a market as a numbering plan, not a country. Zimbabwe, South Africa, Kenya and Nigeria do not agree about digit counts, trunk prefixes, or how many digits a mobile number has. A single "is this a phone number" predicate is a bug waiting to happen; the predicate should take the market as an argument.

The other two drift defects in the same week

The same class produced two more defects worth recording, because they show the shape of the problem.

A children's home was filed as a prospect (§7) — shape valid, meaning wrong. And in an integration with a spreadsheet, a column header said Next Action Required while the code wrote the previous conversation into it, and the column beside it, labelled Phone, received the next action. Every value was present. Every value was in the wrong place. The failure was invisible until a human read the sheet and noticed that a phone number said "Send 7-Day Free Trial".

Three instances, one class. The detector is always the same: **ask what a human will do with this value, and check the value against that use.** A phone number will be dialled. A next action will be followed. A lead will be called. Any validation that does not end in the human's action is testing the wrong thing.

The cost, stated honestly

Normalisation cost a day of work and a permanent addition to the test suite. Recording these incidents in this book costs something else: it tells a reader that the system that produced a confident statement page also produced an undialable phone number in the same week. I have left it in because the alternative — a book about governance whose own record of governance is curated — would be a worse product than the bug.

> **Rule eight.** Validate against what a human will do with the value, not against what the value looks like.

PART THREE

The Rules

CHAPTER 9

Never hold the money

A rule that is written down and not enforced is a preference. This part of the book is about three commitments that are enforced — in code, in copy, and in the shape of the company — and about what enforcement costs.

The first commitment is the one the book is named for: **the platform never holds, moves, converts or disburses merchant money.**

What enforcement looks like

A promise not to hold money is easy to make and easy to break, because breaking it is incremental. Someone adds a "wallet balance" field because it would be convenient. Someone adds a payout button because a merchant asked. Someone adds a currency conversion because a buyer is in another country and the merchant wants the sale. Each step is reasonable, and after four steps you are a payments company with a compliance obligation you did not plan for and a merchant's money inside your failure domain.

So the rule is enforced structurally, in four ways.

No code path exists. There is no debit, credit, conversion or disbursement operation anywhere in the codebase. Not disabled, not feature-flagged: absent. A reviewer can search for the verbs and find nothing, which is a stronger guarantee than a policy that says "do not use the payout module".

The test suite asserts the absence. A test fails if a balance-like concept appears in the settlement arithmetic.

This is unusual and slightly absurd — testing for something that is not there — and it is the only way to make the absence survive a future contributor who has not read this book.

The price list contains no percentage. This is the commercial half of the same commitment. A platform that takes a share of volume has an incentive to see volume, and the way to see volume is to hold it. Flat fees keep the incentive structure clean.

The documentation states what an agency is told. The single question a reseller will ask is whether reselling the rail makes them a money-services business. The answer the architecture supports is no: the buyer pays the merchant, the merchant's own rail records receipt, the platform compares. No party in the chain holds funds, so no party in the chain inherits custody. That paragraph exists as a file in the repository, not as something I remember to say.

The honest cost

Refusing custody means the platform cannot prove payment, cannot earn float, and cannot be the checkout. I set those out in Chapter Two and I will not soften them here. What I will add is the fourth cost, which is social: refusing custody makes the product less impressive.

An investor understands a payments company immediately. "We do not touch the money; we produce the document that proves where it went" requires a sentence of explanation and then a question about why anyone would pay for it. The answer — because the merchant's month-

end costs her more than your fee — is true and unglamorous, and unglamorous truths are hard to fundraise on.

That is a real constraint on this kind of business, and it is worth naming because it explains why the category is underserved rather than merely unexplored. The gap is not an oversight. It is a consequence of who is willing to build what.

What the merchant gets

Against those costs, four things.

Her money never enters a third party's failure domain. If the platform dies, her cash is where it was on Friday: in her wallet, her account, her drawer. She loses a document, not a month.

Her records survive the platform. The statement is recomputable from an export; the export is a CSV she already has. This is a promise most SaaS companies cannot make in good faith, and it is only possible because the platform never became the custodian of anything irreplaceable.

Her accountant can verify the company's work without trusting the company. Arithmetic beats assurance.

Her fees do not scale with her success. A good month costs the same as a quiet one, which means the platform's share of her upside is zero and its incentive is to keep her rather than to grow her transaction count.

The three sentences, again

The rule about money produces a rule about language, and the language rule is the one that will be tested daily.

Matched is not paid. Expected is not owed. To chase is the actionable number.

Every one of those sentences exists because the tempting alternative is a better demo. A statement that says "paid" is more satisfying to look at than one that says "matched", and the satisfaction is borrowed against a lie. A dashboard that presents outstanding amounts as debts makes the numbers bigger and the merchant's anxiety higher. The three sentences are the price of keeping the document honest, and they are cheap at the price.

When a merchant asks you to hold the money

She will. A merchant who trusts you will ask you to take the payments, because it would be simpler for her, and the request will be sincere and reasonable.

The answer that works is not "we do not do that". It is a demonstration: here is your statement, here is the export, here is the accountant's copy; the money went to your wallet, which is why it arrived instantly and why it will still be there if we disappear. Then, if she still wants a checkout, point her at a processor that pays her directly — and keep producing the document that reconciles it.

You are not refusing to help. You are refusing to become a risk she has not agreed to carry.

> **Rule nine.** Enforce the absence of custody in code, in tests, in the price list, and in the paragraph you give an agency. A preference that lives only in your head will not survive your successor.

CHAPTER 10

Never lie in the copy

The second commitment is about language, and it is harder to enforce than the first, because language has no compiler.

The temptation, stated plainly

A merchant's decision to adopt is emotional before it is analytical. She adopts because the morning is exhausting, because she is tired of being treated as a marginal market, because a competitor is moving, because someone finally built for her. Those are legitimate reasons, and they are the reasons a small company can win against a large one that has better distribution and worse empathy.

The problem is that the same lever works when you exaggerate. Scarcity that does not exist. Customers who do not exist. A cost of delay inflated until it is a threat. Shame — you are losing money because you are disorganised — which converts a product decision into a character judgement. Each of these raises conversion in a test and each of them costs something you cannot measure in the same test.

So the commitment has a shape: five levers are permitted, and four moves are forbidden.

The five levers

Righteous resentment, honestly earned. Mainstream providers do not serve these markets — not because they are malicious but because the unit economics do not clear their cost of compliance. Saying so is a factual claim about

the industry, and a merchant who has been ignored recognises it instantly. The honesty constraint is that you may not claim to be discriminated against when you are merely late to a market.

Morning relief. The contrast between a forty-tab panic and a single page is a real, felt, daily experience. It is the most defensible emotional claim in the product because it is the one the merchant can verify before paying.

Territorial urgency. A merchant's competitive position is genuinely local and genuinely first-mover in small markets: the agency that becomes the one whose clients all have order links has a position the second agency does not. This is real scarcity — the scarcity of a position — and it is legitimate to say so. Manufactured scarcity, the countdown timer on a price that will not change, is not.

Ownership and control. No hostage data, no silent bots, no annual lock-in with a six-month exit. For a merchant who has been burned by software before, this is the lever that closes the deal, and it is only available to a platform whose architecture actually keeps the promise.

Reciprocity and status. Give before asking. Teach the thing that helps whether or not she buys. In a market where every seller arrives asking, the one who arrives giving is remembered.

The four forbidden moves

Manufactured scarcity. No fake deadlines, no fake seats, no "three left" counters.

Invented social proof. No testimonials from merchants who have not said those words, no logos of companies who

have not agreed, no "trusted by hundreds" when the number is four.

Buyer shame. Never frame the merchant's current situation as her personal failure. She is running a business in a hard country with the tools she was given. The product exists because the tools were wrong, not because she was.

Emotion overruling arithmetic. This is the subtle one. It means that however good the copy is, the numbers in it must be true, and the price must be what the page says, and the refund must actually happen. Emotional persuasion may not be used to carry a false factual claim past the reader's defences.

Where the rule has to live

A doctrine in a document is a doctrine that lasts until someone writes a landing page in a hurry. So the rule lives in three places: in the doctrine file that anyone generating copy reads first; in the review of every published page; and in the structure of what may be claimed — every quantitative claim in the platform's copy has to be derivable from a value the platform actually holds. Ten shops must be a real count. Seven dollars must be the price the checkout charges. Sixty days must be the length of the trial.

The last of those is worth a sentence. A number in copy is a promise with a date on it, and the most common way a small company lies is by not updating a page when a policy changes. A pricing page is a contract; treat it like one.

What it costs

Doctrine compliance costs conversion. Manufactured scarcity works; that is why the internet is full of it. Honest copy is slower to convert and, in my experience, produces merchants who stay longer and refer more, because the thing they were promised is the thing they got.

I cannot prove that with the sample I have. I can say that the trade is a values choice rather than an optimisation, and that a company which decides it by A/B test will eventually discover the forbidden moves and use them. The only durable defence is to write the constraint into the generator — into the templates, the review checklist and the doctrine file — so that the tired version of you in six months inherits the constraint rather than the temptation.

> **Rule ten.** Remove the instruments of deceit from the writer, not from the reviewer. Reviewers get tired; templates do not.

CHAPTER 11

Never post to your customers

The third commitment is the most unusual, and it is the one I would defend hardest.

Three kinds of people

Tapstall has three classes of person in it, and the distinctions are governance rather than marketing.

The **owner** operates the platform. The **sellers** are the owner's employees, carrying the platform's own offer to prospective merchants. The **operators** are tenants: agencies, accountants' practices, resellers who run the rail for their own clients.

The obvious design is to treat operators as a customer segment and market to them. The correct design is the opposite: **operators are never posted to.** Ever. Not in a campaign, not in a newsletter, not in an onboarding sequence.

Why

An operator's product is the platform's marketing. An agency that resells Tapstall to eleven clients has, in effect, bought the right to be the one who tells those clients what to use. If the platform also emails those clients, or posts to them, it has inserted itself into the operator's relationship and become a competitor to its own customer's business.

The second reason is structural. A tenant's client list is the most valuable asset the tenant has. A platform that can address those clients has, in one careless campaign, conver-

ted a distribution partner into a wary supplier. There is no recovery from that; there is only a contract termination.

A third reason is quieter. Operators are, by nature, the most sophisticated users of the product. They read the page. If the platform's own marketing lands in their inbox alongside the report they are meant to send to their own clients, the pitch they have been making — we keep your affairs in order — takes visible damage.

How it is enforced

Rules about who may be addressed have to be enforced on the **generator**, not on the sent item, because a post that is filtered at the door has already been composed by a system that believed it was allowed.

So the platform's content machinery has an audience parameter drawn from a fixed set — public, buyer, merchant, seller — and operator is not in it. The generator cannot express an operator audience. A template change cannot introduce one, because the value has no representation. And the operator-facing views are partitioned at the query layer, so that one tenant cannot enumerate another's products, items or customers; the partition is not a filter in the template, which is where such filters are usually applied and where they are usually bypassed.

This is a small lattice of relationships and it is worth drawing once:

Surface	Who may be addressed	Who may see
Public	anyone	the platform's own pages
Buyer	one order's counterparty	their order
Merchant	one shop's owner	their shop, their orders, their statement
Seller	the owner's own staff	the queue they are working
Operator	never addressed	their own tenants' data only, partitioned

The rule generalises

Most platforms violate this by accident, because their content pipeline treats every identified person as an audience. The discipline is to separate two questions that are usually conflated: who exists in our system and who may we address. The first is a database. The second is a governance decision, and it should be a closed set.

The same question applies inside the platform, between the owner and the seller. A seller carries the platform's offer; she is not a merchant's counterparty. When she acts on a lead, the record says which seller, and the state machine enforces that one prospect has one voice — because two people from the same company calling the same shop in one week is not just inefficient, it is the exact experience the merchant is escaping.

> **Rule eleven.** Separate who exists from who may be addressed, and make the second list a closed set in the generator.

CHAPTER 12

Two systems, one sheet

This chapter is a field guide, and it exists because integration is where governance goes to die.

Tapstall's sales team works out of a Google Sheet: a CRM with a contact ledger, a tomorrow queue, and two engines already writing into it. Tapstall was to become the third. What follows is what that took, generalised into rules, because the same situation recurs in every company that has ever tried to bolt a new system onto an old one.

Rule one: do not paste over the existing file

The first draft of the integration instructions said: open the Apps Script editor and paste this file. That instruction was wrong, and dangerous, and the person who caught it was the person who owns the sheet, who asked the obvious question: do I paste over the existing code?

The answer is never. Two functions with the same name in one script project do not merge; one silently replaces the other, and the loser is whichever the editor loaded last. In this case both files defined `onOpen`, which builds the menu. The failure mode was a menu that would simply disappear, with no error message anywhere, at some point after a save.

So the new file defines no `onOpen` at all. It registers an installable open trigger instead, under its own name. Two menus, each owned by the file that builds it, and the incumbent's code untouched. It cost one function and it re-

moved a class of failure that would have been diagnosed, wrongly, as a browser problem.

Rule two: claim every shared surface explicitly

An integration is a set of surfaces, and each one must have exactly one owner: the menu, the triggers, each sheet tab, each scheduled job. Ownership must be recorded somewhere a reader can find it, and the integration should refuse to act on a surface it does not own.

The Tapstall file refuses to rebuild the queue tab unless it is explicitly told it owns it, because the incumbent script already rebuilds that tab from every row of the ledger — including, therefore, Tapstall's rows. Two writers clearing and rewriting one view produce flicker at best and loss at worst. The right move was to leave the queue to its existing owner, and to say so in the menu ("only if the other engine stops owning it") so that a future reader knows why nothing happens when they click it.

Rule three: namespace your rows

A shared table is a shared namespace, and the only durable way to share it is to make each writer able to prove which rows are its own. Tapstall's rows carry `tapstall-<id>` in the identifier column. Before editing anything, the integration re-reads the identifiers and filters. A sort, a filter, or a manual reshuffle in the sheet can no longer cause it to write into somebody else's row.

There is a corollary: make the identifier survive the incumbent's deduplication. The incumbent script deduplicates on ID, email and website. A namespaced ID plus a distinct website means the Tapstall rows are never re-added by the

other engine, and the two systems can run for a year without duplicating a merchant.

Rule four: find columns by name, not position

The incumbent adds columns. Any integration that writes to `column 9` will, one Tuesday, write a stage into a phone number. Read the header row at runtime, map names to indices, and fail loudly if a required name is missing.

This is a two-line habit that eliminates the entire class of shifts. It is also, as I discovered, a way to find defects in someone else's code: reading their header row against their writer showed that the queue rebuild wrote twelve columns in a different order than its own header, so that a discussion note landed under Next Action Required and the next action landed under Phone. Every value was present. Every value was in the wrong place. Their morning email was printing the wrong line, in the wrong column, to two people, every day.

Reporting that was more useful than any feature in the release.

Rule five: idempotent both directions

An integration that runs on a timer for a year must be idempotent, or it becomes a machine for generating duplicates and apologies.

Inbound: read the set of identifiers already present, and append only what is not there. Outbound: keep a ledger of the last value successfully pushed per record, and skip anything unchanged. Both directions should also be visible: a sheet tab that lists every push with its HTTP code and the other side's own words when it refused.

That last point matters more than it sounds. When the platform refuses a stage change because the lead is allocated to another seller, the refusal is not an error to be retried. It is the rule working. Retrying it would be the double-pitch the rule exists to prevent, so the integration records the 409 and moves on — and the record of refusals becomes the audit trail of the rule.

Rule six: the sheet holds a key, not the keys

The last rule is about credentials, and it is the one that would matter most after a leak.

The platform's lead surface can be opened by two credentials. The first is the owner's admin secret, which also opens the operations console, the press and the deployment. The second is a key minted for one sheet, which opens the lead feed and the lead actions and nothing else. The sheet gets the second.

The reasoning is unremarkable and routinely skipped: the person running the CRM is not the platform's operator, and should not hold a credential that can deploy code. If the sheet is shared too widely, one scoped secret rotates and nothing else does. This is the integration equivalent of the reserve ladder — a way of arranging permissions so that the least important surface cannot compromise the most important one.

> **Rule twelve.** In any integration with code you do not own: claim surfaces, namespace rows, read headers, be idempotent, and hold a scoped credential.

PART FOUR

The Craft

CHAPTER 13

What broke, and what it taught

The five failures in this chapter are the ones I would put in front of anyone building in this space. They are not the most embarrassing — there are duller defects I have left out — but each one is a class, and the classes are the transferable part.

One: provenance collapse

What happened. A discovery system filed forty-seven records over three weeks. Not one was a business. Every one was a page that mentioned the words.

The signature. Records exist with no field saying where they came from; the queue grows while its actionable subset does not. The system was producing volume without evidence, and volume without evidence always looks like progress from a distance.

The lesson. A record must not outlive the ability to verify it. Every filed lead carries a URL at which its claim can be checked by a person who was not there, and the pipeline refuses anything without one. When we later imported the queue into the CRM, the same rule fired again: the forty-seven rows were counted and excluded rather than imported, because a lead nobody can check is not a lead, it is work in a costume.

The meta-lesson. We deleted the forty-seven only after reading all of them, one by one, and finding the one real record among them. Clean-up is the operation most likely

to destroy the evidence of how the mess arose. The audit has to come first.

Two: budget cannibalisation

What happened. A directory moved and returned a 404. The planner had sorted sources by priority and spent the day's allowance from the top. The pass reported two errors and zero candidates, while the map and the search leg sat unasked with capacity to spare.

The signature. Attempts consumed far exceed results returned, and the deficit is concentrated in one source. Quiet days with working infrastructure.

The lesson. A daily allowance is not a queue. Interleave the sources so that one failure cannot consume the pass, refund a failing job's attempt so the next source gets asked, and cap the refunds so an outage cannot become a crawl. And distinguish refused from answered with nothing: a directory with no matching category is information, an HTTP 404 is a fault, and a system that treats them the same either panics on quiet days or sleeps through outages.

Three: representation drift

What happened. `+234 (0)730 177 1710` became `23407301771710`. A number that could not be dialled, in a queue of numbers that could.

The signature. The value validates and is wrong. It happens wherever a system checks shape instead of meaning: phone numbers, names, addresses, currency amounts, dates.

The lesson. Validate against the action a human will take. A number will be dialled: normalise it to a canonical form, round-trip it, and check the digit count against the market's plan. A name will be called: check whether it is an institution, a charity, a category noun. A next action will be followed: make sure it is in the column labelled next action, which requires reading the header rather than trusting the position.

Four: surface collision

What happened. Two files in one script project both defined the function that builds the menu. One would silently have replaced the other. Separately, two systems both rebuilt the same sheet tab. And, in the other party's code, a writer placed twelve columns in a different order than its own header, so the morning email had been printing the wrong line for as long as it had existed.

The signature. Non-deterministic behaviour in a shared artefact; things appearing and disappearing; two systems both claiming to be the source of one view.

The lesson. Every shared surface gets exactly one owner, recorded where a reader can find it, and each writer proves ownership of its rows structurally rather than by convention. Ambiguity in a shared surface is not neutral — it resolves, invisibly, in favour of whoever writes last.

Five: custody leakage

What happened. Nothing, on the architecture side, and that is the point: the invariant held because it was enforced. The leakage was documentary — a retrospective note that quoted eight-character tails of API tokens as

evidence, in a repository whose whole purpose is to be read.

The signature. A boundary that must not carry something carries a fragment of it. A prompt that must not contain a key contains a key, redacted to four characters. A log line that must not contain a customer's number contains it truncated.

The lesson. Fragments are not safe because they are short. A prefix plus a tail narrows the search space, especially next to a timestamped commit that says "this token worked at 14:52". The rule that survives is the one the audit trail follows: name the class of credential, never a value and never a fragment.

What the five have in common

None of them was a function failure. Each was a failure of a **boundary**: between a claim and its source; between one subsystem's appetite and another's guarantee; between a record and the world; between two authors; between the platform and the merchant's money.

Ordinary testing finds function failures. Boundary failures are found by naming the boundary and asserting something about it — and that is the whole of the method this book is describing. A test suite of 419 assertions is not a virtue in itself. It is a list of boundaries somebody thought to name.

CHAPTER 14

The uncomfortable arithmetic

I want to end with the part that no amount of architecture fixes.

Tapstall is a good piece of software with no paying customers yet. That sentence contains the book's honest centre. Everything in Parts Two and Three is true and running: the storefront, the statement, the gate, the ladder, the doctrine, the integration. And none of it has yet been proved in the only way that counts, which is a merchant paying seven dollars a month for a second month.

Leads are not sales

A discovery system can produce twenty candidates a day. A salesperson can work perhaps fifteen conversations a week, well. The ratio between those numbers is the arithmetic that kills small companies: if your acquisition machinery outruns your human capacity to follow up, you have built a machine for generating guilt.

This is why the daily ceiling on new leads is twenty and not two hundred. It is not a technical limit; it is a statement about how many new names two people can genuinely work. The quota system in Chapter Six protects the order path from the marketing path. The ceiling protects the marketing path from itself.

The first seller is the product

There is a version of this book that ends with a chart going up. This is not that version, because the chart has not started, and because the thing that will start it is not software.

The thing that will start it is one person on a phone, with a checkable hook, in front of a merchant who has forty tabs open. Everything the platform does exists to make that conversation shorter and better: the hook so the first sentence is specific, the link so the demonstration is instant, the statement so the close is a document rather than a promise.

A platform whose discovery finds twenty verified businesses a week and whose sales function calls none of them has not solved distribution. It has automated the part that was never the constraint.

I write this knowing it is the least satisfying chapter for a reader who came for the engineering. But a book about governance which skipped the governance of its own ambition would be a book with a hole in the middle.

What I would tell the next builder

Five things, in the order I would say them.

Pick a document, not a dashboard. The document is the thing that survives, gets forwarded, and makes the sale. Everything else is plumbing to the document.

Refuse the thing you cannot safely hold. Whatever your version of custody is — money, credentials, personal data, an inventory promise — refusing it early is cheaper than unwinding it later, and it becomes a selling point that no competitor can copy without rebuilding.

Write the constraint before the feature. The reserve ladder, the gate, the doctrine and the surface lattice were all written down before there was pressure to break them. That is the only reason they held.

Let the incident into the record. The bugs in Chapter Thirteen are in this book because a company that curates its own history is not auditable, and an unauditable company in a trust business has nothing to sell.

Measure your refusals. A system's honesty is what it declines to do — the lead it would not file, the stage change it would not retry, the copy it would not write, the money it would not hold. Count those, and put the count where you can see it.

The last thing

The shopkeeper with forty tabs will not be saved by a better dashboard. She will be saved by a document that tells her, at seven in the evening, that eleven orders came in, nine matched, two need chasing, and forty dollars is outstanding.

That is a small thing. It is also, in a market where institutions fail visibly and trust is the scarcest input to any transaction, a structural thing: the difference between a business that knows its own position and one that hopes.

Building software for the second kind of business is the work. It is not glamorous, it does not scale on a hockey stick, and it will never be the subject of a thread about a machine that built a company overnight.

It is, however, the work that is left, and it is available to anyone willing to hold three rules: do not touch the money, do not lie in the copy, and do not address the people whose business it is to address others.

Appendix A · The statement specification

A conforming settlement statement for day D and merchant m must satisfy four clauses.

Completeness. It reports expected, matched, to chase, refunded, fees and net; a per-rail breakdown of order counts and outstanding amounts; and one sentence naming the next action.

Derivability. Every figure is computed from the events of day D alone, such that a third party holding the event record and these rules can recompute the statement exactly.

Containment. It contains no script; it carries `no-store`; it is reachable only by a live session of m; an anonymous request is redirected with the shop named; a dead session is told that nothing about the money changed; an unknown shop returns not-found.

Language. It never describes a matched order as paid, an expected order as owed, or the platform's fee as a share of volume. On a day with no orders it states that in words rather than rendering zeros.

An implementation meeting the first three and violating the fourth is non-conforming.

Appendix B · The admission gate

A candidate is filed only if all seven predicates hold.

1. **Not a platform** — not a social network, marketplace, dictionary, news site, bank portal or asset host.
2. **Verified** — the candidate's own page was fetched and read.
3. **Local** — the page names the market, or the candidate came from a bounded map query, or from a directory that is itself local.
4. **Named** — a business name of plausible length, neither a URL nor a sentence.
5. **Not generic** — not a category noun.
6. **Not an institution** — not a public body, bank, university, hospital, school; and not a charity, orphanage, foundation, church, mission, diocese, parish or trust.
7. **Contactable** — a published number valid for the market after normalisation, or an address, or a checkable URL.

Each filed record carries the URL at which its hook can be verified, and the hook itself is a sentence a salesperson can open with.

Appendix C · The reserve ladder

Class	Example key prefixes	Ceiling (of 1,000 writes/day)
Money	`order:`, `store:`, `sale:`, `paid:`, `tx:`, `renew:`	1,000
Security	`lock:`, `pin:`, `rl:`	975
Operator	`audit:`, `img:`, `tpl-ovr:`, `digest:`	900
Automation	`press:`, `mesh:`, `lead:`, `seller:`, `cron:`	800
Telemetry	everything else	700

A write from class p is admitted if writes-spent plus one is at most the ceiling minus the reserves of every class above p. Telemetry is additionally paced across the UTC day. Guarantee: an order write can fail for budget only if all one thousand writes are spent.

Appendix D · The failure taxonomy

Class	Signature	Detector	Response
Provenance collapse	records exist, source absent, queue grows without actionable content	provenance query; fraction of records verifiable at a recorded URL	verified-as-a-predicate; refuse to import unsourced records; count them
Budget cannibalisation	attempts ≫ results, deficit concentrated in one source, quiet days	per-source utilisation; pass-level refund invariant	interleave sources; refund failures; cap attempts; separate quiet from error
Representation drift	value validates, is wrong	round-trip validation; digit count against the market plan; adversarial name corpus	canonical normalisation with refusal; regression tests built from the live failing string
Surface collision	shared artefact changes non-deterministically; two writers, one view	surface register with one owner per surface; owned-namespace identifiers	installable triggers instead of duplicate names; ownership refusal; rows proven by namespace
Custody leakage	a boundary carries a fragment of what it must not	absence tests in the money surface; credential-pattern scan in the repository	structural absence of money operations; name the credential class, never the value

Appendix E · The three commitments

1. **Never hold the merchant's money.** No code path debits, credits, converts or disburses. Fees are flat. The document proves where the money went; the platform is not a party to it.
2. **Never lie in the copy.** Five levers — earned resentment, morning relief, real territorial position, ownership, reciprocity — and four forbidden moves: manufactured scarcity, invented social proof, buyer shame, emotion overruling arithmetic.
3. **Never address the people whose business it is to address others.** Operators are never posted to. The audience of a generator is a closed set; operators are not in it.

A short reading list

For the argument: Sarasvathy, Effectuation (2008), on building from means at hand; Baker and Nelson on bricolage in Administrative Science Quarterly (2005). For the craft of reliability: Beyer et al., Site Reliability Engineering (2016), and Oliner and Stearley on what system logs say (DSN, 2007). For the record: Ijiri, Theory of Accounting Measurement (1975), and Grigg's triple-entry accounting proposal (2005). For the boundary object: Star and Griesemer, in Social Studies of Science (1989). For the case method: Yin, Case Study Research and Applications (2018). For the regulatory frame around custody: the EU e-money directive (2009/110/EC) and the EU AI Act's oversight provisions (2024/1689).

Colophon

Set in Iowan Old Style (text) and system sans-serif (apparatus), rendered from a single Markdown source into HTML and print-ready PDF. The case system described here was, at the time of writing, deployed and versioned 1.9.1, with 419 passing assertions against a 14,855-line codebase, on free-tier infrastructure, in four markets. No merchant's money was held at any point in its construction.

Jacaranda Labs Press · Harare · First edition, October 2026 · CC BY 4.0